Evaluating Commit Message Quality for Early Bug-Inducing Change Detection

Pavlo Havrylyuk
College of Engineering
Oregon State University
havrylyp@oregonstate.edu

Brad Goodlett
College of Engineering
Oregon State University
goodletb@oregonstate.edu

Jace Bolante
College of Engineering
Oregon State University
bolantej@oregonstate.edu

***Abstract*—As software systems grow in complexity, detecting defects early in the development lifecycle has become increasingly critical. While existing tools leverage large language models to automate pull request reviews, they remain prohibitively expensive. We propose a lightweight classifier that evaluates commit message quality as a low-cost first-pass filter, flagging commits likely to introduce defects before escalating them to more resource-intensive analysis tools. This tiered approach aims to reduce the overall cost of defect detection without sacrificing the depth of review that complex changes demand.**

I. Introduction

Software development teams rely on version control systems like Git, SVN or Mercurial to manage code changes, with commit messages serving as the primary documentation of why changes were made [1]. Despite the availability of Large Language Model (LLM) based code review tools capable of identifying defects in pull requests, these solutions carry significant financial and computational costs that make them impractical for continuous use [2]. This research investigates whether quantifiable commit message quality correlates with bug introduction rates across real-world repositories.

Our key insight is that the quality of a commit message (how well it describes what changed and why) may reflect the care and rigor with which the underlying code change was made. By mining open-source repositories with known bug-inducing commits, we can train a lightweight classifier that uses quantifiable commit message quality metrics to predict whether a commit is likely to introduce a bug. This approach requires no code analysis, making it fast enough to run in real-time as a first-pass filter before escalating flagged commits to more expensive LLM-based review tools.

There are many research projects focusing on commit quality and bug detection at the commit content level [3] [4] [5] [6]. Existing research seems to focus mostly on the specific changes in the commit themselves, or aim to predict a quality score for a commit message. Our aim is to determine whether the quality of the message has any bearing on the bug introduction rate of commits. Focusing solely on the commit message allows for real-time feedback on commits without expensive computation.

First, to develop our model, we need to define our two research questions:

RQ1: Does commit message quality correlate with bug introduction rates?

RQ2: Which commit message characteristics are the most predictive of bug introduction?

To evaluate the first question, we need to find whether any meaningful relationship exists between commit message quality and bug-inducing commits. We evaluate this by measuring the correlation between our structural and semantic quality features and the known bug-inducing labels in our dataset.

Assuming that the first research question is in fact true, then we can investigate the second question of which specific features contribute most to predicting whether a commit will introduce a bug or not. By analyzing the feature weights of our trained SVM classifier, we can identify which structural properties, such as message length, use of imperative mood, or number of external references, and which semantic signals carry the most predictive performance. Understanding which features matter most provides insight into what distinguishes bug-inducing commits from clean ones from just the commit message.

To evaluate our model, our research utilizes two datasets to use as ground truth references to test our proposed method against. First, we use Mozilla's regressors-regressions dataset [7] [8] that links bug introducing commits to their corresponding fix commits. We use these commit hashes to identify known bug-inducing changes and then sample additional non-buggy commits from the same repositories. Second, we use the BugHunter dataset [9] which similarly has commit hashes which contain bugs.

We scrape the repositories that each dataset contains hashes for, and label the commits using the datasets to create an expanded dataset containing the commit metadata. We will continue to refer to our expanded datasets as Mozilla and BugHunter datasets respectively, as they are what we base the entirety of our expanded data on.

For each commit message, we extract a feature vector capturing both structural quality signals, such as title length, capitalization, and number of external references, and semantic content through Term Frequency-Relevance Frequency (TF-RF) word embeddings. We train a Support Vector Machine (SVM) classifier on these features, using K-Fold cross vali-

dation to tune model parameters and identify which commit message characteristics are most predictive of bug-inducing changes. We rely on the Linear SVM's learned numeric feature coefficients and TF-RF coefficients to determine the relevance of each feature and word/word pair.

We will evaluate our approach based on two parts. First, we assess within dataset performance by training and testing each model on the same dataset using K-Fold cross-validation SVM regularization parameters. This measures whether commit message features carry enough signals to distinguish bug inducing commits, from the clean ones in a open source software project. Second, we assess cross-dataset performance by training on one dataset and testing on another. Here we measure whether the learned patterns transfer across to a different project.

We use the F1 score as our primary evaluation metric because of the balance between precision and recall. We additionally report per-label F1 scores to assess whether the models are biased toward one class, and we analyze the learned feature weights of each SVM to determine the relative contribution of the ten structural quality features versus the TF-RF word embeddings.

II. Background and Motivation

A. Background

Open Source Software (OSS) development projects are constantly evolving through hundreds of commits made by countless different developers weekly, and for some projects even daily. The volume of commits being made can be difficult for reviewers and maintainers to keep up with. Fortunately software developers have come up with many different tools to automatically assess the quality of the commit, this can range from test suites and static analysis tools to Continuous Integration (CI) pipelines and LLM tools [10]. However many of these automated tools choose to ignore the intent/communication from the developer and rely on the maintainers and reviewers to observe that data manually. While this is a solution that does work, we assert that the information learned from commit messages can be efficiently automated using different parts of the commit message. An example of this, consider two commits with unknown changes to the code, one has the commit message *"Fixed null pointer exception in user authentication when token is expired"* and the other has the commit message: *"Fixed some bugs"*. One of the commit messages is short, vague, and general while the other is detailed and explicitly says what it is trying to do, based on just this information, which of these seem less likely to introduce a bug to the program?

B. Motivation

In large development teams, hundreds of commits may be pushed in a single day, making it infeasible to subject every commit to deep automated analysis. Commit messages are generated with every commit and provide an immediate, zero-cost signal about the nature and quality of a change, without requiring any code analysis. If commit message quality correlates with bug introduction rates, a lightweight classifier could serve as a real-time first-pass filter, flagging only the commits that warrant deeper, more expensive review. This would allow teams to benefit from the thoroughness of static analysis tools or a LLM-based tools while dramatically reducing their overall cost.

III. Related Work

There are two main fields of work relating to our research: Defect prediction and natural language processing.

A. Defect Prediction

Defect prediction can be split into two different subfields that relate to our work: Just-In-Time Software-Defect-Prediction (JIT-SDP) models and defect prediction with support vector machines (SVM). Just-In-Time prediction models "leverage software change histories to predict potential defect-inducing software changes as soon as they are committed into the repository" [11]. Zhao et al. conducted a study on the effectiveness of JIT-SDP and found that the current JIT models have improved over early models, however they seem to plateaued recently [11]. Alnagi et al. conducted a survey on the most effective JIT-SDP models and evaluation metrics, and found that Logistic Regression and Random Forest are the two most common Machine Learning models used for JIT and that F1 Score was the most common metric because of "...the imbalance(d) nature of datasets used for JIT-SDP. Since the number of instances labeled as not-defective is much larger than the ones labeled as defective." [12]. The research conducted in these papers is different from ours because they conducted surveys relating to JIT while we are attempting to create a technique that similar to JIT. The novelness of our research is that JIT considers the code differences in the commit as well as other information such as the developer and sometimes commit related semantics such as the commit message where as our technique will only use the commit related semantics.

Support Vector Machines have been used in various research in defect prediction, Gray et al. used static code metrics to train a SVM for defect prediction within a repository. However, their research was conducted at the module level and not at the commit level as our research is [13].

1) Commit Checker: The "Commit Checker" paper aims to research whether it is possible to create a JIT prediction model to predict whether code changes contain a bug. Unlike most research, the authors are attempting to create a pre-commit prediction model that can stop bug-inducing commits before they are actually committed. [14]

The authors create a pre-commit Git hook that extracts several features from the code changes, then passes them into a machine learning model that predicts whether the changes introduce a bug or not. The authors train three machine learning models using Random Forest, Decision Tree, and Logistic Regression. In their research, the Logistic Regression model performed the best with around 80% accuracy. Using

this model, they created a VS Code extension that makes the prediction and warns the developer that the current changes have a high risk level.

Similar to our research, the Commit Checker paper aims to create a lightweight prediction model that runs before the commit can reach the remote repository. Unlike our research, however, the Commit Checker project aims to predict based on the code changes rather than the commit message itself.

2) DeepJIT: The DeepJIT paper researches whether a deep learning based JIT software defect prediction model can effectively predict whether code changes introduce a bug or not. The paper asks the following four research questions:

- How effective is DeepJIT compared to the state-of-the-art baseline?
- Does the proposed model benefit from both commit message and the code changes?
- Does DeepJIT benefit from the manually extracted code changes features?
- What are the time costs of DeepJIT?

DeepJIT uses Convolutional Neural Networks to attempt to capture deeper meaning from code change data. This model is trained on the commit changes as well as the commit message. [15]

The DeepJIT paper is similar to our research in that it utilizes the commit message as part of its prediction model. The research is different because it utilizes the code changes as part of its input data as well. The paper does seek to confirm whether the model benefits from having the commit changes and message, which is similar to our research question which is asking whether the commit message has any importance in the ability to predict whether a commit introduces a new bug.

B. Natural Language Processing

The field of natural language processing can also be split into three separate subfields: Commit message analysis, commit message generation, and term frequency embeddings. Many studies have been conducted analyzing the quality of commit message through GitHub or other Version Control Systems. Chahal et al. found ten rules and 11 attributes to measure the quality of commit messages, including commit message length, capitalization, imperative mood, and more [16]. Tian et al. found that many commit messages did not meet their standard of a "good commit message", and concluded that many of the messages correctly stated "what" was changed but did not state "why" it was changed [17].

There are many different kinds of Term frequency embeddings, including Term Frequency-Inverse Document Frequency (TF-IDF), Term Frequency-Relevance Frequency (TF-RF), word2vec, and BiDirectional Encoder Representations from Transformers (BERT). Hericko et al. used TF-IDF to assess commit messages and assign them into one of the three groups: adaptive, corrective, and perfective [18]. Our research will use TF-RF, the largest difference between TF-IDF and TF-RF is that TF-RF is supervised while unsupervised. Originally TF-RF was proposed by Lan et al. as an alternative weighting scheme to TF-IDF, they found that it outperformed TF-IDF

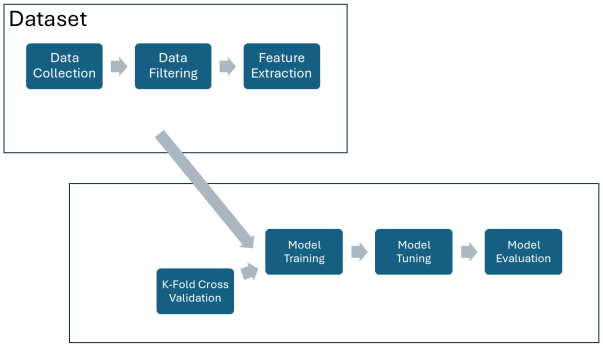

Fig. 1.

and other term weighting approaches in most cases [19]. Sarwar et al. conducted a similar study to Hericko et al. by using BERT to assess commit messages and assign them one of: adaptive, corrective, or perfective [20]. The approach by Sarwar et al. is also unsupervised and has the same differences from our research as the research conducted by Hericko et al.

Commit Message Generation is a subfield that focuses on automatically creating commit messages based on the code change. While this subfield does not directly relate to our research, it is indirectly related. If the code difference can be used to generate a commit message, and the code difference can be used to predict the risk of a defect being added, it stands to reason that a commit message generated from the code difference would be able to.

Additionally, there are some research papers that relate to both the fields of natural language processing and defect prediction. Li et al. asked how commit message quality impacted software defect proneness. They used Window Quality Score to estimate the message quality and used Welch's t-test to analyze the Window Quality Score, and found that at each different window size, there was a statistically significant difference [21]. Li et al. noted finding how commit message quality impacted software defect proneness was not their primary target, and because of that they did not build a full defect prediction model using "state-of-the-art features identified in defect prediction literature." Barnett et al. studied whether commit message length or commit message content could be used to add explanatory power to JIT-SDP models [22].

IV. Approach

A. Data Filtering

We applied the same filtering pipeline to both the Mozilla and the BugHunter datasets to ensure consistency. For the Mozilla dataset, we scraped commit messages from Mozilla's Mercurial repository using the bug-inducing commit hashes provided by the regressors-regressions dataset [7] [8], and randomly sampled an equal number of commits not referenced as bugs. For the BugHunter dataset [9], we collected commit messages from 14 of the 15 GitHub repositories it referenced,

and sampled non-buggy commits from the repositories. Table I summarizes the raw and filtered statistics for both datasets.

After collection, we applied several filters to both datasets. First, we removed default or generic commit messages such as "Initial Commit" or "Update Foo.md", as well as messages containing only whitespace, since these carry no meaningful quality signal. Next, we removed commits authored by bots (dependency update bots, CI/CD tool bots), identified through author metadata and known bot naming patterns. We also removed merge commits, as their messages tend to describe the merge itself rather than the underlying changes. Squash commits were removed where present for similar reasons. Finally, we stripped bug ID references (e.g., "Bug 123123") from commit messages to prevent the model from learning project-specific metadata rather than message quality signals; as shown in Table I, these references existed in the Mozilla dataset but nearly absent from BugHunter. After balancing each dataset so that the buggy and clean classes are equal in size, the filtered datasets serve as the input to our feature extraction pipeline.

TABLE I
RAW DATASET STATISTICS AND COMMITS REMOVED BY EACH FILTERING STEP.

	Mozilla	BugHunter
Projects	1	15
Raw commits	105,556	465,318
Buggy commits	52,778	8,052
Clean commits	52,778	457,266
Merge commits removed	1,865	68,939
Bot commits removed	1,830	72
Squash commits removed	0	365
Bug ID references found	93,427	3

B. Feature Engineering

After filtering the datasets, we transform each commit message into a feature vector with 11 different dimensions that capture both the textual semantics and commit message quality signals. The first feature will use Term Frequency-Relevance Frequency (TF-RF) to represent the individual words in each message as numeric value. TF-IDF (Term Frequency-Inverse Document Frequency) is the more common word embedding technique than TF-RF. However TF-IDF is used for unsupervised learning which makes it not relevant to our use case. Before applying TF-RF we will clean the dataset further by removing all punctuation, numbers and capitalization as this can cause issues with the embedding. Additionally the TF-RF term matrix will capture both unary and binary word embeddings, also known as single words and 2 word pairs. One of the models is trained only on single words.

Research by Chahal et al. found eleven features that define a good commit message [16]. We incorporate seven of their features and define three additional features of our own, shown in Table II. The length of the description combines two of the features defined by Chahal et al. (commit description existence and number of paragraphs in the description), and the commit hash features add another type of reference that can be tracked.

TABLE II
COMMIT MESSAGE QUALITY FEATURES USED IN OUR MODEL. FEATURES 1–7 ARE ADAPTED FROM CHAHAL ET AL. [16]; FEATURES 8–10 ARE DEFINED BY US.

#	Feature	Source
1	Length of summary	[16]
2	First character of summary is capitalized	[16]
3	Imperative mood in summary	[16]
4	Number of external references in summary	[16]
5	Number of external references in description	[16]
6	Number of filename references in summary	[16]
7	Number of filename references in description	[16]
8	Length of description	Ours
9	Number of commit hashes in summary	Ours
10	Number of commit hashes in description	Ours

C. Model Construction and Training

We trained a supervised classification using the engineered features described in the previous subsection. While conducting research with TF-IDF, Aggarwal et al. found that Support Vector Machine (SVM) and Logistic Regression models performed the best [23]. Naeem et al. found that a Support Vector Machine was the most efficient in their study [24]. Because of the similarities between TF-IDF and TF-RF we decided to also use a SVM as SVM's excel when evaluating high-dimensional features spaces created by the TF-IDF and TF-RF will create a similar feature space. Additionally it allows for analysis of the individual features weights to assess whether the features are effective or not.

We tuned the SVM regularization parameter C through K-Fold cross validation and selected the best performing C value for our final model. K-Fold cross validation will be used to increase reliability and reduce the risk of overfitting.

V. EVALUATION

A. Dataset

Our datasets are built using labels from two open source datasets. We use data from Mozilla's Regressors-Regressions dataset [7] [8], alongside data from the BugHunter dataset [9].

1) Mozilla Dataset: The Mozilla dataset is structured to combine bug-fix and bug-inducing commit ids together. We iterate over all datapoints in this dataset and extract the bug-inducing commit ids.

We then use the public repository to collect commit metadata for around 100k datapoints. We use the bug-inducing commit hashes during commit metadata extraction to create a balanced final dataset, ensuring a nearly 50/50 split between bug-inducing and non-bug commits. We label the commits as bug-inducing if their hash matches one from the Mozilla dataset, and non-bug-inducing if it does not.

Our final dataset collected from the Mozilla dataset contains 105,556 data points. 52,778 of these commits are labeled as bug-inducing, and 52,778 of them are labeled as non-bug-inducing.

In our data filtering process, we found that this dataset contains an enormous amount of commit message that start with "Bug" followed by some numerical identifier. Many of

TABLE III
PROJECT NAMES PAIRED WITH THE NUMBER OF BUG HASHES COLLECTED FROM EACH REPOSITORY IN THE BUGHUNTER DATASET. [9]

Project Name	Number of Bugs
Android-Universal-Image-Loader	84
BroadleafCommerce	863
MapDB	195
antlr4	134
ceylon-ide-eclipse	399
elasticsearch	4280
hazelcast	3899
junit	67
mcMMO	414
neo4j	731
netty	1395
orientdb	975
oryx	89
titan	120

the 93,427 commits starting with this prefix also contained a reviewer's name in the commit message, giving a direct indication to the quality of the commit.

2) BugHunter Dataset: The BugHunter dataset is similar to the Mozilla dataset in that it contains commit hashes that may or may not contain bugs. We extract any commit hash in the dataset where "Number of Bugs" is greater than 0.

We then collect all commit metadata from the referenced GitHub repositories to be further processed. We label all commits as bug-inducing if their hash matches one collected from the BugHunter dataset, and non-bug-inducing if it does not. After our collection process, we have 8,052 commits labeled as bug-inducing, and 457,266 commits labeled as non-bug-inducing. This imbalance is corrected during our data processing step where the 457,266 non-bug-inducing commits will be filtered and randomly sub-sampled to select an even dataset of bug-inducing and non-bug-inducing commits.

This dataset is much more realistic in its commit messages, with only 3 commits that specifically start with the same "Bug" followed by a numeric identifier. Because it is spread across 14 different projects, it is much more likely to be representative of real world commit messages. There were 68,939 merge commits that we filtered out.

B. Metrics

Evaluation of our model's performance will be done using the F1 score, a combination of the precision and recall of the model. The equation for F1 score is as follows:

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

where TP is the number of true positives (bug-inducing commits correctly flagged), FP is the number of false positives (clean commits incorrectly flagged), and FN is the number of false negatives (bug-inducing commits that were missed).

This gives us a range between 0 and 1 that we can use to evaluate the true accuracy of the model. F1 score was chosen as our evaluation metric because it takes into consideration both precision and recall.

We also analyze the LinearSVM Feature Coefficients on each trained model to determine which of the features were most important to each model, and which words each model picked up on as most relevant.

C. Experiment Procedure

The pipeline of our experiment procedure can be broken down into five different steps:

1. Collect raw commit data
2. Filter commits
3. Balance dataset
4. Feature extraction
5. Model training

Each step has a python script associated with it, except for the first step which has two scripts, one for each dataset. Both of the scripts collected the commit data from GitHub, however the BugHunter dataset was made up of multiple repositories which required a different implementation than the Mozilla dataset. Next, the filtering script, filters out all the bot commits, merge commits, and generic commits using Regex. The balance script finds the class label (buggy or non-buggy) with less occurrences and reduces the class label with more occurrences to be equal to it.

The feature extraction script extracts both the numeric features and the TF-RF features from the balanced commit file. We used Regex to find the external references, commit hashes, and filename references. We calculate imperative mood using the Spacy Python library by checking if the verbs in the given string are in the infinitive form. We find the TF-RF features using a custom extended implementation of the Scikit-Learn TfidfVectorizer and TfidfTransformer classes [25]. The TfrfVectorizer and TfrfTransformer implement the weighting scheme proposed by Lan et al. over the IDF implementation while inheriting the preprocessing features [19].

The model training script uses the Scikit-Learn SVM class [25] to train a linear SVM on the combined numeric features and TF-RF features. Our script contains three different functions for training models: one that trains and tests a model on the same dataset, one that trains a model on the one dataset and tests it on another, and one that trains multiple models with different 'C' values for the SVM to find the optimal one. The scripts calculates and outputs multiple metrics such as the precision, recall, and F1-Scores for each class label, a confusion matrix, the numeric feature weights, and the top positive and negative TF-RF feature weights.

D. Results

To evaluate our technique we trained 3 different models, one with the Mozilla dataset, and two with the BugHunter dataset, one with unigrams and bigrams, and one with just unigrams. Our results for each of the models are shown in Table IV, with each model being tested on both of the datasets. The models were fairly successful at testing on their own dataset, with an F1-Score of above 0.60 for the BugHunter dataset models, and

TABLE IV
A TABLE CONTAINING THE F1-SCORE AND F1-SCORE PER LABEL FOR EACH OF THE THREE TRAINED MODELS TESTED ON EACH OF THE TWO DIFFERENT DATASETS.

Training Dataset	Testing Dataset	F1-Score	F1-Score on label=0	F1-Score on label = 1
BugHunter (1-2 word pairs)	BugHunter	0.65	0.66	0.64
BugHunter (1-2 word pairs)	Mozilla	0.47	0.59	0.27
BugHunter (single word)	BugHunter	0.62	0.63	0.61
BugHunter (single word)	Mozilla	0.46	0.56	0.30
Mozilla	BugHunter	0.50	0.03	0.66
Mozilla	Mozilla	0.89	0.88	0.89

TABLE V
A TABLE CONTAINING THE ABSOLUTE VALUE OF THE FEATURE WEIGHTS FOR EACH NUMERICAL FEATURE FOR EACH OF THE THREE TRAINED MODELS

Feature Name	BugHunter (1-2 word pairs)	BugHunter (single word)	Mozilla
Length of Summary	0.1342	0.0667	0.0483
Length of Description	0.0778	0.0862	0.056
First Letter of Summary Capitalized	0.0264	0.0275	0.3559
Imperative Mood in Summary	0.0158	0.0012	0.0159
Commit Hashes in Summary	0.0073	0.0040	0.2506
Commit Hashes in Description	0.0775	0.0837	0.2686
External References in Summary	0.0827	0.0839	0.0166
External References in Description	0.1272	0.1296	0.0684
Filename References in Summary	0.0079	0.0043	0.0283
Filename References in Description	0.1036	0.1047	0.2263

an F1-Score of almost 0.90 for the Mozilla model. However the models did not test well on the opposite dataset, with all three of the models scoring at or below 0.50 F1-Score.

The F1-Score for each model by label is also available in the table. An interesting observation with the label data is when the training dataset was the same as the testing dataset, then the F1-Score between the two labels was similar. However when trained on one dataset and tested on the other, the F1-Scores for each label were vastly different. The BugHunter models were accurate in predicting the non-buggy commits in the Mozilla dataset but not the buggy ones, and the Mozilla model was more accurate in predicting the buggy commits than the non-buggy commits by a large margin (0.66 to 0.03).

Table V contains the feature weights for each numeric features that we used in our models. There is not a large amount of consistency between the weights used by the models. The BugHunter models both contain some similar weights (External references in the description, filenames in description) but other are vastly different (length of summary). Only the Mozilla model had any features weights that were above 0.20, with the two BugHunter models only having 5 total feature weights above 0.10. This shows that the TF-RF matrix tended to strongly outweigh the numerical features that we included in our model.

VI. DISCUSSION

Our results show that commit message quality features, when combined with TF-RF word embeddings, can predict bug-inducing commits with moderate to high accuracy when evaluated within the same project. However, the sharp performance decrease in cross dataset evaluation reveals limitations in what the models actually learned during training.

A. Key Limitations

The numeric quality features defined in Table II, had limited influence on model predictions relative to the TF-RF word embeddings. Only the Mozilla model assigned weights above 0.30 to its structural feature, and the BugHunter models had weights not exceeding 0.14 as seen in Table V. This suggests that, the current features we have selected, do not capture enough distinct signals to have a meaningful output on our models performance. It is possible that more carefully engineered features could improve performance, but we did not explore these in the current study.

Another limitation of our experiment was that we did not choose to filter "stop words". "stop words" are words such as "the" or "and" that do not actually contribute any context to the embedding, in future experiments it would be best to filter the stop words to better find the intent for each sentence. Additionally, while SciKit-Learn Python library [25] provides a list of english stop words that can be used, it would be ideal to create a custom list of stop words which are specific to commit message generation.

B. Threats to Validity

The primary threat to internal validity is overfitting. The strong within-dataset performance along side with the poor cross-dataset scores suggests that our models may have memorized dataset specific patters rather than learning generalizable relationships between message quality and defect introduction. Although we used K-Fold cross-validation to train the SVM regularization parameter, cross validation within a single data set did not reduce overfitting.

Our analysis of "commit message quality" through the ten structural features in Table II and TF-RF embeddings may not fully capture what developers mean by quality. Important features such as the clarity of the rationale for a change, the

accuracy of the description relative to the actual code diff, or the presence of contextual information for reviewers are not represented in our feature set. A better commit message quality model might yield different results.

Our evaluation was limited to two data set ecosystems: Mozilla Firefox (a large, mature C/C++ project with strict commit conventions) and the BugHunter dataset (a collection of Java project). These two data sets only represent a narrow slice of the open source landscape. Projects written in other languages, using different version control workflows, or are maintained by a smaller team, may have different relationships between commit message quality and defect rates. The poor cross-dataset results reinforce this concern and suggest that any practical deployment of this technique would require retraining on project specific commits.

The models trained on the BugHunter dataset have a higher likelihood of variation in model performance due to the large imbalance of buggy and non-buggy commits. There were over 50 times the number of non-buggy commits, this could lead to the non-buggy words being learned by the TF-RF Vectorizer to be drastically different through repeated use. To counteract this we set the random generation to the same state each time for consistent results.

C. Future Direction

There were several avenues for future research that emerged from our findings.

The most immediate improvement would be to preprocess commit messages more aggressively, stripping project-specific metadata (bug IDs, reviewer names, component tags, "Signed-off-by" lines) before feature extraction. This would force the model to rely on the semantic content and structural quality of the message rather than metadata that correlates with historical bug reports.

Future work should explore commit message quality features that go beyond surface level structure. Some feature that could be explored include readability metrics, alignment between commit message and actual code diff as well as incorporating the "what vs. why" distinction identified by Tian et al. [17] as a measurable feature could also improve predictive power.

If commit message quality does carry genuine predictive signals, this technique could be integrated into CI/CD pipelines as a lightweight prefilter. Commits flagged by the message quality classify could be escalated to more expensive analysis tools (static analyzers, LLM based reviewers), while high-quality messages pass through with reduced scrutiny. Evaluating the cost-effectiveness of such a tiered system in a production environment would be valuable practical contribution to this research.

Our study used TF-RF with a linear SVM, but other combinations remain unexplored. Other classifiers such as Random Forest and Naive Bayes have shown promise in related defect prediction work and may interact differently with the high dimensional features produced by term-frequency weighting.

D. Replication Package

All scripts for dataset collection, dataset filtering, feature extractions, and model training are publicly available in our GitHub repository. [1]

VII. Conclusion

This paper investigated whether commit message quality, measured through structural features and TF-RF word embeddings, can predict bug-inducing commits. We trained SVM classifiers on two datasets (Mozilla's regressors-regressions dataset and the BugHunter dataset) and evaluated both within-dataset and cross-dataset performance.

Our results provide a mixed answer to our two research questions. For the first question "whether commit message quality correlates with bug introduction rates?" the evidence is conditionally affirmative. Within a single repository, commit message content carries meaningful signal: the Mozilla-trained model achieved an F1 score of 0.89, and the BugHunter models reached 0.62-0.65 on their respective datasets. However, this signal is largely repository-specific. All three models degraded substantially when tested on the opposite dataset, with cross-dataset F1 scores at or below 0.50, indicating that the learned patterns do not transfer across project contexts.

For the second question "Which commit message characteristics are most predictive?" we found that the TF-RF word embeddings consistently dominated the ten structural quality features we defined in Table II. The structural features proved to be insufficient to help the classification on their own. This suggests that the specific vocabulary used in commit messages, rather than their structural form, carries the stronger predictive signal. Although this vocabulary tends to reflect project-specific conventions (reviewer names, component identifiers, subsystem keywords) rather than universal markers of commit quality.

These findings suggest that commit-message-based classifiers are most viable as repository-specific tools. Within a single project, a lightweight SVM trained on commit message features could serve as a low-cost first-pass filter, flagging commits with message patterns historically associated with bug introductions before escalating them to more expensive analysis. However achieving cross-project performance, will require more aggressive preprocessing to strip repository-specific metadata, exploration of alternative embeddings and model architectures, and training on a more diverse dataset. We hope this work encourages further research into the underexplored relationship between developer communication and software defect prediction.

References

[1] Stack Overflow, "Stack overflow developer survey 2022," https://survey.stackoverflow.co/2022/, 2022, accessed: 2025.

[2] W.-X. Ouédraogo *et al.*, "Cigar: Cost-efficient program repair with llms," 2024. [Online]. Available: https://arxiv.org/abs/2402.06598

[3] A. Tavakkoli Barzoki, "Framework for bug inducing commit prediction using quality metrics," Master's Thesis, Western University, 2024. [Online]. Available: https://ir.lib.uwo.ca/etd/10159

[1] https://github.com/bolantej/CommitQualityForBugPrediction

[4] M. R. Islam and M. F. Zibran, "Sentiment analysis of software bug related commit messages," in *Proceedings of the 27th International Conference on Software Engineering and Data Engineering (SEDE)*, New Orleans, Louisiana, USA, 2018, pp. 1–6. [Online]. Available: https://cs.uno.edu/~zibran/resources/MyPapers/SentimentBug_SEDE2018.pdf

[5] D. Faragó, M. Färber, and C. Petrov, "A full-fledged commit message quality checker based on machine learning," in *Proceedings of the 47th IEEE International Conference on Computers, Software, and Applications (COMPSAC)*, 2023. [Online]. Available: https://faerber-lab.github.io/assets/pdf/publications/Commit-Message-Quality-Checker_COMPSAC2023.pdf

[6] J. Li and I. Ahmed, "Commit message matters: Investigating impact and evolution of commit message quality," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 806–817. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00076

[7] M. Castelluccio, "mozilla/bugbug," Jan. 2024. [Online]. Available: https://github.com/mozilla/bugbug

[8] F. Petrulio, D. Ackermann, E. Fregnan, G. Calikli, M. Castelluccio, S. Ledru, C. Denizet, E. Humphries, and A. Bacchelli, "Szz in the time of pull requests," 2022.

[9] R. Ferenc, P. Gyimesi, G. Gyimesi, Z. Tóth, and T. Gyimóthy, "Bughunter dataset," *Mendeley Data*, 2020. [Online]. Available: https://data.mendeley.com/datasets/8tx7kjbkg4/2

[10] G. Dolcetti and E. Iotti, "A dual perspective review on large language models and code verification," *Frontiers in Computer Science*, 2025. [Online]. Available: https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1655469/full

[11] Y. Zhao, K. Damevski, and H. Chen, "A systematic survey of just-in-time software defect prediction," *ACM Comput. Surv.*, vol. 55, no. 10, Feb. 2023. [Online]. Available: https://doi.org/10.1145/3567550

[12] E. Alnagi and M. Azzeh, "Just-in-time software defect prediction techniques: A survey," in *2024 15th International Conference on Information and Communication Systems (ICICS)*, 2024, pp. 1–6.

[13] D. Gray, D. Bowes, N. Davey, Y. Sun, and B. Christianson, "Using the support vector machine as a classification method for software defect prediction with static code metrics," in *International Conference on Engineering Applications of Neural Networks*, 2009. [Online]. Available: https://api.semanticscholar.org/CorpusID:15019643

[14] N. Z. Z. Oishie and B. Roy, "Commit-checker: A human-centric approach for adopting bug inducing commit detection using machine learning models," in *Proceedings of the 15th Innovations in Software Engineering Conference*, ser. ISEC '22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: https://doi.org/10.1145/3511430.3511463

[15] T. Hoang, H. Khanh Dam, Y. Kamei, D. Lo, and N. Ubayashi, "Deepjit: An end-to-end deep learning framework for just-in-time defect prediction," in *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, 2019, pp. 34–45.

[16] K. K. Chahal and M. Saini, "Developer dynamics and syntactic quality of commit messages in oss projects," in *International Conference on Open Source Systems*, 2018. [Online]. Available: https://api.semanticscholar.org/CorpusID:49238094

[17] Y. Tian, Y. Zhang, K.-J. Stol, L. Jiang, and H. Liu, "What makes a good commit message?" in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 2389–2401. [Online]. Available: https://doi.org/10.1145/3510003.3510205

[18] T. Heriko, S. Brdnik, and B. umak, "Commit classification into maintenance activities using aggregated semantic word embeddings of software change messages," in *SQAMIA*, 2022. [Online]. Available: https://api.semanticscholar.org/CorpusID:252919535

[19] M. Lan, C. L. Tan, J. Su, and Y. Lu, "Supervised and traditional term weighting methods for automatic text categorization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 4, pp. 721–735, 2009.

[20] M. U. Sarwar, S. Zafar, M. W. Mkaouer, G. S. Walia, and M. Z. Malik, "Multi-label classification of commit messages using transfer learning," in *2020 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, 2020, pp. 37–42.

[21] J. Li and I. Ahmed, "Commit message matters: Investigating impact and evolution of commit message quality," in *Proceedings of the 45th International Conference on Software Engineering*, ser. ICSE '23. IEEE Press, 2023, p. 806–817. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00076

[22] J. G. Barnett, C. K. Gathuru, L. S. Soldano, and S. McIntosh, "The relationship between commit message detail and defect proneness in java projects on github," in *2016 IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR)*, 2016, pp. 496–499.

[23] D. Aggarwal, V. Bali, A. Agarwal, K. Poswal, M. Gupta, and A. Gupta, "Sentiment analysis of tweets using supervised machine learning techniques based on term frequency," *Journal of Information Technology Management*, vol. 13, pp. 119–141, 01 2021.

[24] M. Naeem, F. Rustam, A. Mehmood, M. zzud din, I. Ashraf, and G. S. Choi, "Classification of movie reviews using term frequency-inverse document frequency and optimized machine learning algorithms," *PeerJ Computer Science*, vol. 8, p. e914, 03 2022.

[25] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.